

# Report of Reading Theme Retrieval Agent

May 22, 2026

## 1 Introduction

This project implements a command-line AI agent called Reading Theme Retrieval Agent. The goal is to explore how an AI agent can use Information Retrieval methods to find useful context before generating an answer. The agent is built around a small thematic collection of women-centered literary works.

Instead of answering directly like a normal chatbot, the agent first retrieves relevant information from local reading notes, a theme guide, recommendation rules, and user memory. The retrieved context is then passed to an OpenAI-compatible language model API. The system also includes a memory function and a confidence mechanism to improve the reliability of the agent's answers.

## 2 Dataset

The local dataset consists of three text files in the documents folder. The first file, `book_notes.txt`, contains short notes for a small collection of 10 books, including *A Room of One's Own*, Kim Ji-young, *Born 1982*, *The Handmaid's Tale*, *Little Women*, *My Brilliant Friend*, *Cat's Eye*, and *The Color Purple*. Each book entry includes a title, author, themes, and a short note.

The second file, `theme_guide.txt`, explains the thematic labels used in the project, such as friendship, sisterhood, family, mother-daughter relationship, marriage, motherhood, and so on.

The third file, `recommendation_rules.txt`, gives simple rules for how the agent should make recommendations. For example, the agent should prefer books that appear in the retrieved notes, explain recommendations based on retrieved themes, and warn the user when the retrieved context is weak.

The working system also uses a local memory file, `memory_store/memory.json`. This file stores user-provided notes added through the `remember` command.

## 3 Method

The system is organized as a small, modular command-line agent. The main interface is implemented in `main.py`, while `agent.py` controls the agent workflow. Retrieval, memory management, and the LLM call are separated into `retriever.py`, `memory.py`, and `llm_client.py`.

The core IR method is TF-IDF retrieval with cosine similarity. Local documents and saved memories are converted into searchable chunks. When the user enters a query, `TfidfVectorizer`

represents both the query and chunks as vectors, and cosine similarity is used to rank the most relevant chunks.

The agent supports several commands. The search command only performs retrieval, while ask retrieves context first and then calls a Berget.AI model through an OpenAI-compatible API. API settings are stored in a local .env file, while .env.example is included as a safe template.

The project also includes a confidence-aware retrieval mechanism. The system first computes a numeric confidence score from the highest TF-IDF cosine similarity score. Scores above 0.25 are treated as strong, scores between 0.10 and 0.25 as weak, and lower scores as unreliable.

During testing, I found that the numeric score alone was not always sufficient, because unrelated questions could still retrieve general files such as theme\_guide.txt or recommendation\_rules.txt. To address this, I added a source-aware check: for book-related questions, the final context is only considered reliable when the retrieved sources include book\_notes.txt. The output therefore shows the numeric score, base score strength, final context strength, and retrieved source names.

## 4 Working with AI Tools

During development, I used ChatGPT to help establish the initial structure of the Python files and to write and format the README file. Although using AI tools accelerated the development process, I still needed to manually verify the implementation of TF-IDF retrieval, the storage and loading mechanisms for memory items, the configuration of the OpenAI-compatible API, and how the retrieved context is passed to the language model. Therefore, this project was not only a practical exercise in using AI tools to complete programming tasks but also an opportunity to critically refine AI-generated output.

## 5 Limitations

This project is a small prototype. The book collection is limited and manually created, so the agent cannot support broad literary analysis. The retrieval method is based on TF-IDF, which is simple and useful for lexical matching, but it does not capture deeper semantic relationships as well as embedding-based retrieval.

The memory system is also simple. It stores user notes in a JSON file and retrieves them with the same TF-IDF method, but it does not yet summarize or organize long-term memory. In the future, the project could be extended with semantic embeddings, more books, richer metadata, web search, visualization of retrieval scores, conversation compaction, or other mechanisms.

## 6 Links

- GitHub repository: [https://github.com/Thisisruiii/reading\\_theme\\_retrieval\\_agent\\_\\_](https://github.com/Thisisruiii/reading_theme_retrieval_agent__)
- Demo video: [https://1drv.ms/v/c/6b679df81c78177f/IQD1VU5GqpE8TJKHIEDxv3e4AVWv4g\\_EGvOXbg1bH6SgU38?e=dd0gBC](https://1drv.ms/v/c/6b679df81c78177f/IQD1VU5GqpE8TJKHIEDxv3e4AVWv4g_EGvOXbg1bH6SgU38?e=dd0gBC)